# Zero-Trust Network Architecture with a Service Mesh

Implementation of a Zero-Trust Network Architecture in Kubernetes using Service Mesh Technologies

## Introduction

Traditional network security relies heavily on perimeter defense, assuming that internal network traffic is inherently trustworthy. In modern, highly distributed microservices architectures, this flat-network approach leaves systems vulnerable to lateral movement if a single container is compromised. This project introduces a Zero-Trust architecture within a Kubernetes environment. By utilizing a service mesh, the system ensures that no communication is trusted by default, enforcing strict identity-based routing and mutual TLS (mTLS) encryption for all internal traffic, thereby elevating the security posture of cloud-native applications.

## Problem Statement

Default Kubernetes networking allows all pods to communicate with each other unrestrictedly. While this simplifies initial development, it presents a critical security flaw in production environments. Malicious actors or compromised dependencies can easily exploit this to access sensitive backend databases or internal APIs. Configuring robust, encrypted, and strictly authorized inter-service communication manually is error-prone and requires extensive operational overhead, highlighting the need for an automated, mesh-driven approach to internal network security.

## Scope Of Work

- The project automates and enforces internal network security for a containerized application stack. Core modules include:
- Microservices Deployment: Containerization and deployment of a multi-tier application (Frontend, API, Database) to serve as the testing environment.
- Service Mesh Integration: Implementation of a lightweight service mesh to manage pod-to-pod traffic without altering application code.
- Network Policy Enforcement: Configuration of Container Network Interface (CNI) rules to physically isolate namespaces and specific workloads.
- Traffic Encryption: Automated provisioning and rotation of cryptographic certificates for mutual TLS (mTLS) across all services.

## Objectives

### Primary Objectives

- Zero-Trust Implementation: Deploy a functional Kubernetes cluster where all default inter-pod communication is denied.
- Automated mTLS: Implement a service mesh (e.g., Linkerd) to automatically encrypt all traffic between microservices.
- Granular Access Control: Develop strict network policies ensuring services can only communicate with their explicitly defined dependencies.

### Secondary Objectives

- Traffic Observability: Integrate visual dashboards to monitor network topologies and identify unauthorized connection attempts in real-time.

### Technologies

| Layer | Technology | Role |
|---|---|---|
| Backend/App | Go / Node.js | Microservices logic and API endpoints |
| Database | PostgreSQL | Isolated backend data store |
| Infra | Kubernetes (k3d/kind) | Container orchestration and runtime |
| Networking | Linkerd | Service mesh for mTLS and traffic routing |
| Security | Calico CNI | Enforcement of Layer 4 network policies |
| Observability | Prometheus & Grafana | Scraping and visualizing network and network metrics |

### Expected Outcomes

A fully functional Kubernetes cluster demonstrating a hardened, Zero-Trust network.

Measurable metrics showing successful interception and encryption of all internal traffic.

A comprehensive architectural blueprint for securing cloud-native applications, highly relevant for modern infrastructure engineering roles.